Zero-Trust Compute Sandbox & Multi-Tenant Security Spec

CodeForge Platform

Engineering Specification

Document 2 of 4

Version	1.0
Status	Draft for Security Review
Date	July 2026
Authors	CodeForge Engineering
Classification	Confidential
Distribution	Engineering, Security, Compliance

CONFIDENTIAL — For internal use and B2B client security reviews.

Revision History

Version	Date	Author	Description
1.0	2026-07-11	Engineering	Initial specification. Grounded in existing `SECURITY.md`, `judge0.conf`, `docker-compose.yml`, and codebase analysis.

Contents

1 Purpose and Audience **4**

2 Threat Model **4**
2.1 Threat Actors 4
2.2 Trust Boundaries 5

3 Execution Sandbox Architecture **6**
3.1 Current Implementation: Judge0 with Isolate 6
3.2 Network Segmentation 7
3.3 Multi-Tenant Enhancement: Per-Tenant Resource Quotas 7
3.3.1 BullMQ Rate Limiting per Organization 7
3.3.2 Judge0 Per-Submission Limits 8
3.3.3 Concurrent Submission Caps 8

4 Cross-Tenant Data Isolation **10**
4.1 Application-Layer Isolation (Existing) 10
4.2 Database-Layer Isolation (RLS) 10
4.3 Object Storage Isolation 10
4.4 Redis Queue Isolation 11

5 Authentication and Authorization Security **12**
5.1 Staff Authentication 12
5.1.1 Multi-Tenant Enhancement: Token Binding 12
5.2 Candidate Authentication 13
5.3 Role-Based Access Control 13

6 Scoring and Integrity Guarantees **14**
6.1 Server-Authoritative Scoring 14
6.2 Exam Timer Integrity 14
6.3 Proctoring and Integrity Score 14
6.4 Plagiarism Detection 15

7 Input Validation and Abuse Prevention **16**
7.1 Request Validation 16
7.2 Rate Limiting 16
7.3 Output Truncation 16
7.4 HTTP Security Headers 17

8 Compliance Readiness Matrix **18**

9 Security Architecture for Cloud Metadata Protection **19**

10 Internal Database Connection Protection **19**

11 Known Security Gaps and Remediation Plan **20**

12 Summary of Security Controls by Layer **21**

Purpose and Audience

This document provides a comprehensive technical security specification for the CodeForge multi-tenant B2B platform, with a particular focus on the isolation of untrusted code execution across different business clients. It is written to serve two audiences:

1. **Internal engineering teams** implementing the multi-tenant transition described in Document 1.
2. **B2B client security teams** conducting vendor security assessments during procurement. This document is designed to answer the questions typically found in CAIQ (Consensus Assessments Initiative Questionnaire) and SOC 2 pre-qualification reviews.

All references to source files, configuration values, and architectural decisions in this document are grounded in the CodeForge repository as of July 2026.

Threat Model

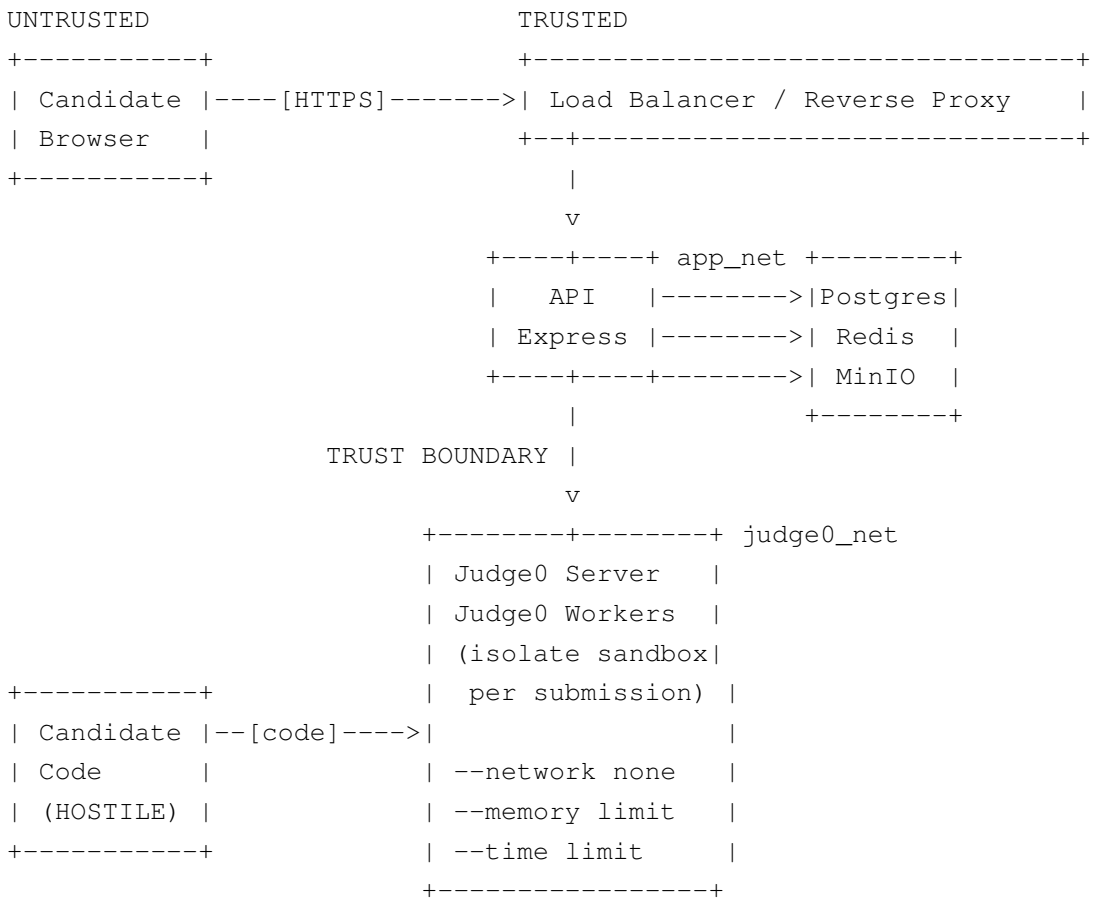
CodeForge accepts and executes arbitrary source code submitted by untrusted external users (job candidates). This is an inherently adversarial workload. The platform must defend against the following threat actors and attack surfaces:

Threat Actors

Table 1: Threat Actor Classification

Actor	Capability	Objective
Malicious Candidate	Arbitrary code execution within sandbox	Escape the sandbox, access other candidates' data, exhaust shared resources, reach internal services
Rogue Staff Member	Authenticated API access with recruiter or reviewer role	Access data belonging to a different organization, escalate privileges to super_admin
External Attacker	Network access to public endpoints	Credential stuffing, token theft, API abuse, denial of service
Compromised Dependency	Code execution within the application process	Lateral movement to database, Redis, or object storage

Trust Boundaries



The critical trust boundary sits between the API process and the Judge0 worker pool. Candidate code never executes within any process that has access to the application database, Redis queues, or object storage.

Execution Sandbox Architecture

Current Implementation: Judge0 with Isolate

CodeForge uses Judge0 CE 1.13.1 as its primary code execution engine. Judge0 delegates to `isolate`, a Linux sandbox originally developed for competitive programming contests (IOI). Isolate provides process-level isolation through:

- **Linux Namespaces**: PID, mount, network, and user namespaces isolate each submission from the host and from other concurrent submissions.
- **Cgroups (v1)**: CPU time, wall clock time, memory, and process count are hard-limited per submission.
- **Seccomp profiles**: System call filtering restricts the available kernel interface.

The current configuration in `judge0.conf`:

```
# Execution limits (per submission)
CPU_TIME_LIMIT=5
CPU_EXTRA_TIME=1
WALL_TIME_LIMIT=10
MAX_CPU_TIME_LIMIT=15
MAX_WALL_TIME_LIMIT=30
MEMORY_LIMIT=256000          # 256 MB
MAX_MEMORY_LIMIT=512000      # 512 MB absolute cap
MAX_PROCESSES_AND_OR_THREADS=60

# Security: no network access, no overrides
ENABLE_NETWORK=false
ALLOW_ENABLE_NETWORK=false
```

Network Segmentation

The `docker-compose.yml` establishes two isolated Docker networks:

Table 2: Network Segmentation Matrix

Service	app_net	judge0_net
Postgres (application)		
Redis (BullMQ)		
MinIO		
API / Worker		(bridge)
Web (Next.js)		
Judge0 Server		
Judge0 Workers		
Judge0 DB (private)		
Judge0 Redis (private)		

Only the API and Worker containers bridge both networks. This means that even a complete sandbox escape from an isolate worker lands the attacker in the `judge0_net` segment, which contains only Judge0's own private Postgres and Redis instances. There is no route from `judge0_net` to the application database, the BullMQ Redis, MinIO, or any other application service.

Multi-Tenant Enhancement: Per-Tenant Resource Quotas

In the single-org deployment, all submissions share the global Judge0 worker pool. In the B2B platform, a single enterprise client's assessment could overwhelm shared resources and degrade service for other tenants. The following controls are introduced:

BullMQ Rate Limiting per Organization

```
// jobs/queues.ts -- per-tenant submission rate limiter
import { RateLimiter } from "bullmq";

const ORG_RATE_LIMITS: Record<string, { max: number; duration:
    number }> = {
  standard:  { max: 50,  duration: 60_000 },  // 50 jobs/min
  priority:  { max: 200, duration: 60_000 },  // 200 jobs/min
  dedicated: { max: 0,   duration: 0 },       // unlimited (own
      pool)
};
```

```

export function orgRateLimiter(
  tier: "standard" | "priority" | "dedicated"
): RateLimiter | undefined {
  const cfg = ORG_RATE_LIMITS[tier];
  if (!cfg || cfg.max === 0) return undefined;
  return new RateLimiter(cfg.max, cfg.duration);
}
```

Judge0 Per-Submission Limits

The API layer (`lib/judge0.ts`, function `runCases`) already enforces per-submission CPU and memory limits via the Judge0 batch submission payload. These are clamped to the maximums in `judge0.conf`:

```
// lib/judge0.ts -- existing enforcement (unchanged)
const submissions = opts.cases.map((c) => ({
  source_code: b64(opts.sourceCode),
  language_id: lang.judge0Id,
  stdin: b64(c.input),
  expected_output: b64(c.expectedOutput),
  cpu_time_limit: Math.min(15, Math.max(0.1, opts.timeLimitMs /
     1000)),
  memory_limit: Math.min(512000, Math.max(16000, opts.
     memoryLimitKb)),
}));
```

Even if a tenant's test configuration specifies 30 seconds of CPU time, the Judge0 server rejects it because `MAX_CPU_TIME_LIMIT=15` in `judge0.conf` is a hard ceiling.

Concurrent Submission Caps

A new middleware, applied at the `/api/candidate/{sessionId}/run` and `/api/candidate/{se` endpoints, counts active (queued + running) submissions per organization and rejects new ones beyond the cap:

```
// middleware/executionCap.ts
const CONCURRENT_CAPS = { standard: 20, priority: 100, dedicated:
    Infinity };

export async function enforceExecutionCap(
  req: Request, _res: Response, next: NextFunction
) {
```

```
  const orgId = req.candidate!.test.orgId;
  const tier = await getOrgTier(orgId);
  const cap = CONCURRENT_CAPS[tier];
  if (cap === Infinity) return next();

  const active = await prisma.submission.count({
    where: {
      session: { test: { orgId } },
      status: { in: ["queued", "running"] },
    },
  });
  if (active >= cap) {
    return next(tooManyRequests(
      "Assessment system is at capacity. Please retry in a few
         seconds."
    ));
  }
  next();
}
```

Cross-Tenant Data Isolation

Application-Layer Isolation (Existing)

Every staff-authenticated route in the codebase filters queries by `req.user.orgId`. For example, in `routes/question.routes.ts`, question listings are filtered by:

```
where: { orgId: req.user!.orgId, isArchived: false }
```

Candidate routes are implicitly scoped because the candidate's capability token binds to exactly one `Invitation`, which references exactly one `Test`, which belongs to exactly one `Organization`.

Database-Layer Isolation (RLS)

As specified in Document 1, PostgreSQL Row Level Security policies enforce org-scoped access at the database level. This provides defence in depth: even if a code path omits the `orgId` filter, the database itself refuses to return rows belonging to other tenants.

Object Storage Isolation

The current `lib/s3.ts` uses a single MinIO bucket (`codeforge`) with key-based namespacing:

```
sessions/<sessionId>/snapshot-<timestamp>.jpg
```

In the multi-tenant deployment, the key prefix is extended to include the `orgId`:

```
orgs/<orgId>/sessions/<sessionId>/snapshot-<timestamp>.jpg
```

For enterprise-tier clients, a separate S3 bucket (or MinIO tenant) is provisioned. The S3 client factory selects the appropriate bucket based on the tenant context:

```
// lib/s3.ts -- modified
export function tenantBucket(orgId: string, tier: string): string {
  if (tier === "dedicated") return `codeforge-${orgId}`;
  return config.s3.bucket; // shared bucket, key-prefixed
}

export function tenantKey(orgId: string, key: string): string {
  return `orgs/${orgId}/${key}`;
}
```

Presigned URLs generated by `presignGet` are time-limited (15 minutes, as in the existing code) and scoped to the exact object key. There is no list-bucket permission granted to the application role.

Redis Queue Isolation

BullMQ jobs contain the `orgId` in their payload (as introduced in Document 1). Job processors validate the `orgId` before executing. There is no mechanism by which a BullMQ consumer can "see" jobs belonging to a different tenant, because all jobs share the same queues and are dispatched by the shared worker pool. The tenant boundary is logical, enforced at the data layer, not at the queue level.

For dedicated-tier clients, separate Redis instances (or Redis database indices) are provisioned, and the worker process connects to the appropriate Redis based on a routing table.

Authentication and Authorization Security

Staff Authentication

The existing implementation (`middleware/auth.ts`, `lib/tokens.ts`) uses a two-token pattern:

Table 3: Token Architecture

Token	Lifetime	Storage	Security Properties
Access Token (JWT)	15 minutes	Client `localStorage`	Short-lived, self-contained, contains `orgId` and `role`. No database lookup required for validation.
Refresh Token	30 days	`httpOnly` `SameSite=Lax` cookie, path-scoped to `/api/auth`	Opaque 48-byte random value. Only the SHA-256 hash is stored in the `RefreshToken` table. Revocable per-session.

Multi-Tenant Enhancement: Token Binding

In the multi-tenant deployment, the JWT access token is bound to both the `orgId` and the tenant's domain. The tenant resolution middleware (Document 1, Section 4) cross-checks the JWT's `orgId` against `req.tenant.orgId`. A token issued on `acme.codeforge.com` is rejected if presented on `initech.codeforge.com`.

Refresh token cookies are scoped to the tenant's subdomain via the `Domain` attribute:

```
res.cookie("refreshToken", raw, {
  httpOnly: true,
  secure: config.isProd,
  sameSite: "lax",
  path: "/api/auth",
  domain: req.tenant?.slug
    ? `.${req.tenant.slug}.${config.platformDomain}`
    : undefined,
  maxAge: config.refreshTokenTtlDays * 86400 * 1000,
});
```

Candidate Authentication

Candidate tokens are 256-bit (32-byte) cryptographically random values generated by `crypto.randomByte` in `lib/tokens.ts`. They are:

- **Bound to one invitation**: The token is a `@unique` column on the `Invitation` table.
- **Expiring**: Each invitation has an `expiresAt` timestamp, enforced server-side in `middleware/candida`
- **Revocable**: Setting `invitation.status` to `"revoked"` immediately invalidates the token.
- **Lockable**: After 5 failed identity confirmations (`failedAttempts`), the link is locked.
- **Cross-tenant safe**: The middleware cross-checks `invitation.test.orgId` against `req.tenant.orgId` (as specified in Document 1).

Role-Based Access Control

The existing RBAC system (`middleware/auth.ts`) defines three roles:

Table 4: RBAC Permission Matrix

Action	super_admin	recruiter	reviewer	Notes
Create/edit questions				`requireStaffWrite`
Create/edit tests				`requireStaffWrite`
Send invitations				`requireStaffWrite`
Override scores				`requireStaffWrite`
View results				`requireStaffRead`
View integrity data				`requireStaffRead`
Manage org settings				Planned: `requireRole("super_admin")`
Manage users				Planned: `requireRole("super_admin")`

In the multi-tenant deployment, the `super_admin` role is strictly org-scoped. A super_admin of Org A has zero visibility into Org B's data. Platform-level administration (tenant provisioning, billing, support) is handled by a separate **platform_admin** role that is not exposed to tenant users.

Scoring and Integrity Guarantees

Server-Authoritative Scoring

All score computation happens server-side. The architecture explicitly prevents clients from influencing scores:

1. **DSA questions**: The grading worker (`services/grading.ts`) submits source code to Judge0, receives verdicts, and computes a weighted score based on test-case pass rates. No client-supplied score is accepted.
2. **MCQ questions**: The `finalizeSession` function in `services/session.ts` scores MCQ answers by comparing selected option IDs against the server-stored answer key (from `typeConfig`). The client never receives answer keys.
3. **Hidden test cases**: The `toCandidateQuestion` function in `services/session.ts` strips all hidden test cases (`isHidden: true`) from the candidate-facing response. Candidates see only sample cases.

Exam Timer Integrity

The session deadline is server-authoritative:

- The `endsAt` field on `TestSession` is set when the candidate starts the exam.
- A BullMQ delayed job (`scheduleAutoSubmit` in `jobs/queues.ts`) fires at `endsAt + 3 seconds` to auto-submit.
- A repeatable sweep job (`ensureSweepJob`, every 60 seconds) catches sessions whose delayed job was lost (e.g., Redis restart).
- Client-side timers are purely cosmetic. A candidate who manipulates the client timer still has their session auto-submitted server-side.

Proctoring and Integrity Score

The integrity system (`services/integrity.ts`) computes a 0–100 score based on weighted proctoring events:

Table 5: Proctoring Event Penalties (from `integrity.ts`)

Event Type	Severity	Penalty	Cap	Notes
`tab_switch`	2	4	40	Browser tab changed
`fullscreen_exit`	3	6	30	Exited fullscreen mode
`large_paste`	3	8	24	Derived server-side from editor deltas
`face_absent`	2	4	20	Webcam heuristic (not biometric)
`paste_blocked`	2	3	15	Attempted paste
`devtools_suspected`	3	5	15	DevTools detection signal
`webcam_denied`	3	10	10	Declined webcam access
`window_blur`	1	1	8	Window lost focus
`copy_blocked`	1	1	5	Attempted copy
`multi_monitor`	2	3	3	Multiple displays detected

Critical design decision: the `large_paste` event is **derived server-side** from the editor event delta stream (`EditorEventBatch` model), not reported by the client. A tampered client cannot suppress this signal.

Plagiarism Detection

The plagiarism engine (`services/plagiarism.ts`) implements a winnowing-based fingerprinting algorithm:

1. Source code is tokenized (identifiers $\rightarrow$ `V`, numbers $\rightarrow$ `N`, strings $\rightarrow$ `S`; keywords and punctuation preserved).
2. k-gram hashes ($k = 5$) are computed and winnowed ($w = 4$) to select representative fingerprints.
3. Starter code fingerprints are subtracted to avoid false positives from provided templates.
4. Pairwise comparison uses max-containment similarity: $\text{sim} = \max\left(\frac{|\text{shared}|}{|\text{A}|}, \frac{|\text{shared}|}{|\text{B}|}\right)$.
5. Reports above 15% similarity are stored with character-level matched ranges for side-by-side review.

In the multi-tenant context, plagiarism scans are scoped to a single test's submission cohort. Submissions from Tenant A are never compared against submissions from Tenant B, even if both tenants use the same question (via shared question pools).

Input Validation and Abuse Prevention

Request Validation

All request bodies are validated using Zod schemas (in `@codeforge/shared` and in `middleware/vali` JSONB configuration fields (`typeConfig`, `settings`, `brandingConfig`) are re-validated against per-type Zod schemas before persistence.

Rate Limiting

The existing `middleware/rateLimit.ts` defines four tiers:

Table 6: Rate Limit Configuration (from `rateLimit.ts`)

Limiter	Window	Limit	Applied To
`authLimiter`	15 min	30	`/api/auth/*`
`identityLimiter`	15 min	20	Candidate identity confirmation
`executionLimiter`	1 min	30	`/run`, `/submit` endpoints
`globalLimiter`	1 min	600	All endpoints

In the multi-tenant deployment, rate limits become **per-tenant** by keying on `req.tenant.orgId` rather than the default IP-based keying:

```
export const tenantGlobalLimiter = rateLimit({
  windowMs: 60 * 1000,
  limit: 600,
  keyGenerator: (req) => req.tenant?.orgId ?? req.ip ?? "unknown",
  standardHeaders: true,
  legacyHeaders: false,
});
```

Output Truncation

Program output from Judge0 is truncated to 16 KB per stream (`stdout`, `stderr`) in `lib/judge0.ts`:

```
function truncate(s: string | null, max = 16_384): string | null {
  if (s == null) return null;
  return s.length > max ? `${s.slice(0, max)}\n... [truncated]` :
      s;
}
```

This prevents storage abuse and ensures that a malicious submission producing gigabytes of output does not exhaust database storage.

HTTP Security Headers

The `helmet` middleware in `app.ts` sets standard security headers including `X-Content-Type-Options`, `X-Frame-Options`, `Strict-Transport-Security`, and `Content-Security-Policy`. CORS is restricted to the configured `webOrigin`; in the multi-tenant deployment, the allowed origin is dynamically computed from the tenant's domain.

Compliance Readiness Matrix

This section maps CodeForge's security controls to common compliance frameworks referenced in B2B procurement.

Requirement	Framework	CodeForge Control
Data isolation between tenants	SOC 2 CC6.1	PostgreSQL RLS policies, org-scoped application queries, S3 key-prefix isolation, network segmentation
Access control enforcement	SOC 2 CC6.3	RBAC middleware (`requireRole`), JWT with embedded `orgId`, cross-tenant token binding
Encryption in transit	SOC 2 CC6.7	TLS termination at load balancer; internal Docker networks are isolated; Judge0 API accessible only within `judge0_net`
Encryption at rest	SOC 2 CC6.1	PostgreSQL transparent data encryption (configurable); MinIO server-side encryption (SSE-S3)
Audit logging	SOC 2 CC7.2	`VoiceLog` equivalent for all grading/session jobs (structured pino logs); `ProctoringEvent` table; planned audit log table
Vulnerability in dependencies	SOC 2 CC7.1	`npm audit` in CI pipeline; Dependabot alerts
Sandboxed code execution	Custom (client)	Isolate sandbox (namespaces, cgroups, seccomp), `ENABLE_NETWORK=false`, network segmentation, output truncation
Resource exhaustion prevention	Custom (client)	Per-submission CPU/memory/PID limits, per-tenant BullMQ rate limits, concurrent submission caps, Judge0 global maximums
Metadata service protection	Cloud (SSRF)	`ENABLE_NETWORK=false` in Judge0; `-network none` in Docker executor; no route from sandbox to `169.254.169.254`

Security Architecture for Cloud Metadata Protection

When CodeForge is deployed on cloud infrastructure (AWS EC2, GCP GCE, Azure VM), candidate code must be prevented from accessing the instance metadata service at `169.254.169.254`. This is a common SSRF vector that could expose IAM credentials.

Threat

A malicious candidate submits code that makes an HTTP request to `http://169.254.169.254/latest/meta-data/iam/security-credentials/` to steal the instance's IAM role credentials.

Control

Layer 1: `ENABLE_NETWORK=false` in `judge0.conf`. Isolate creates the submission process in a network namespace with no interfaces. All network system calls fail.

Layer 2: The Docker executor fallback uses `--network none`, which removes all network devices from the container.

Layer 3: Judge0 runs exclusively in the `judge0_net` Docker network, which has no route to the host's `169.254.169.254`. Even if isolate's network namespace were bypassed, the Judge0 container itself cannot reach the metadata endpoint.

Layer 4 (recommended for cloud deployments): Configure the cloud provider's IMDSv2 (AWS) or equivalent to require a PUT-based token handshake, which is not possible from a container with no network access.

Internal Database Connection Protection

Threat

A sandbox escape allows a candidate's process to make TCP connections within the Judge0 container. The attacker attempts to connect to the application Postgres on port 5432.

Control

Layer 1: Isolate's network namespace prevents all outbound connections from the sandboxed process.
Layer 2: The Judge0 container itself is on `judge0_net`. The application Postgres is on `app_net`. These are separate Docker networks with no peering. DNS resolution for `postgres` (the application database hostname) fails inside `judge0_net`.
Layer 3: The Judge0 container does have access to `judge0-db`, its own private Postgres. This database contains only Judge0's internal state (submission metadata, language configs). It holds zero application data, zero user credentials, and zero candidate submissions beyond the input/output pairs currently being graded.

Known Security Gaps and Remediation Plan

The following gaps are documented in the existing `SECURITY.md` and are tracked for remediation:

Table 8: Known Gaps and Planned Remediation

Gap	Risk	Current Mitigation	Planned Fix
Access token in `localStorage`	XSS could steal token	Sanitized Markdown is the only user-authored HTML; 15-min expiry	Move to in-memory only storage; add CSP `script-src` restrictions
Candidate token stored raw in DB	DB breach exposes valid tokens	Tokens are revocable; DB is encrypted at rest	Hash tokens at rest once email re-send is implemented
No structured staff audit log	Limited forensic capability	Structured pino logs capture grading and session events	Dedicated `AuditLog` table with actor, action, target, timestamp
Judge0 unauthenticated internally	Lateral movement risk within `judge0_net`	Port 2358 not published beyond localhost; network segmentation	Enable `AUTHN_TOKEN` in `judge0.conf` and mirror to API config

Summary of Security Controls by Layer

Table 9: Defence-in-Depth Security Stack

Layer	Controls
Network perimeter	TLS termination, CORS, Helmet headers, rate limiting
Application layer	Zod validation, RBAC middleware, tenant resolution, JWT binding, output truncation
Database layer	RLS policies per tenant, parameterized queries (Prisma), encrypted-at-rest
Queue layer	Per-tenant priority, org rate limits, concurrent submission caps
Object storage	Key-prefix isolation, time-limited presigned URLs, no list-bucket permission
Execution sandbox	Isolate (namespaces, cgroups, seccomp), `ENABLE_NETWORK=false`, network segmentation, per-submission resource limits
Operational	Structured logging, proctoring audit trail, plagiarism scanning, sweep jobs, health checks

End of Document 2. Continues in Document 3: Extensible White-Labeling & Configuration Schema.